BİLGİSAYAR PROGRAMLAMA

HAFTA -2 ÖRNEKLERİ

örnek1: Kaynak kod: algoritmanın bilgisayara yazılmış hali

```
def kinetic_energy(m_kg, v_ms): # adım 1: girdi
   if m_kg <= 0: # adım 2: denetim
      raise ValueError("Kutle pozitif olmali")
   return 0.5 * m_kg * v_ms**2 # adım 3: hesap
E = kinetic_energy(2.0, 3.0)
print(E) # adım 4:
```

ÇIKTI

9.0

#Örnek2: Serbest Düşme

```
# kullanıcıdan yükseklik al, sayıya çevir
h = float(input("Yükseklik (m): ")) # 20
g = 9.81
# h = ½ g t² → t = √(2h/g)
t = (2 * h / g) ** 0.5
v = g * t
Ek_birim_kutle = 0.5 * v**2
print(f"Düşme süresi: {t:.2f} s")
print(f"Çarpma hızı: {v:.1f} m/s")
print(f"kg başına Ek: {Ek_birim_kutle:.0f} J")
```

ÇIKTI

Yükseklik (m): 20

Düşme süresi: 2.02 s

Çarpma hızı: 19.8 m/s

kg başına Ek: 196 J

#Örnek3:Ohm Yasası

```
# DC devre: float yeterli
V = 12.0
R = 240.0
I = V / R # 0.05 A
P = V * I # 0.6
("--- DC devre ---")
print(f"Akım I = {I:.3f} A")
print(f"Güç P = {P:.3f} W")
```

ÇIKTI

Akım I = 0.050 A

Güç P = 0.600 W

#Örnek4: AC seri RL devresi:

```
# AC seri RL devresi: complex gerekli
import cmath
R, XL = 30.0, 40.0 # ohm
Z = complex(R, XL) # (30+40j)
print("\n--- AC seri RL devresi ---")
print(f"Empedans Z = {Z} ohm")
print(f"|Z| = {abs(Z):.1f} ohm") # 50.0
faz = cmath.phase(Z) # 0.927 rad
print(f"Faz açısı = {faz:.3f} rad = {faz * 180 / cmath.pi:.1f} derece")
I = 220 / Z # (2.64-3.52j)
print(f"Akım I = {I:.2f} A")
```

```
print(f"|I| = {abs(I):.1f} A") # 4.4 A
print(f"I gerçek, sanal = {I.real:.2f} A, {I.imag:.2f} A") # bileşenler
```

ÇIKTI

```
--- AC seri RL devresi ---
Empedans Z = (30+40j) ohm
|Z| = 50.0 ohm
Faz açısı = 0.927 rad = 53.1 derece
Akım I = 2.64-3.52j A
|I| = 4.4 A
I gerçek, sanal = 2.64 A, -3.52 A
```

#Örnek5: Eğik atış

```
import math

v0, aci, g = 20.0, 30.0, 9.81
th = math.radians(aci)

# Menzil, Maksimum Yükseklik, Uçuş Süresi
R = v0**2 * math.sin(2 * th) / g
H = (v0 * math.sin(th))**2 / (2 * g)
T = 2 * v0 * math.sin(th) / g

print(f"Menzil (R): {R:.2f} m")
print(f"Tepe Yüksekliği (H): {H:.2f} m")
print(f"Uçuş Süresi (T): {T:.2f} s")
```

ÇIKTI

```
Menzil (R): 35.31 m
Tepe Yüksekliği (H): 5.10 m
```

Uçuş Süresi (T): 2.04 s

#Örnek6:Karşılaştırma ve Mantık: Sigorta / Gerilim Kontrolü

```
I = 13.5 # ölçülen akım (A)
sigorta = 16.0  # sigorta değeri (A)
asiri = I > sigorta   # False

print(type(asiri)) # <class 'bool'>
V = 235.0
V_min, V_max = 207.0, 253.0
guvenli = V_min <= V <= V_max

if asiri or not guvenli:
   print("Devreyi kes!")
elif I > 0.9 * sigorta:
   print("Uyarı: sınıra yakın")
else:
   print("Normal")
```

ÇIKTI

```
<class 'bool'>
Normal
```

#Örnek7: Birikimli Atama ile Sayısal İlerleme

```
a = 2.0     # m/s^2
dt = 0.5    # s
v = 0.0
x = 0.0
t = 0.0
```

```
# 1. Adım
v += a * dt
x += v * dt
t += dt

# 2. Adım
v += a * dt
x += v * dt
t += dt

x_tam = 0.5 * a * t**2  # Analitik çözüm

print(f"Sayısal x: {x:.2f} m")
print(f"Analitik x: {x_tam:.2f} m")
```

ÇIKTI

```
Sayısal x: 1.50 m
Analitik x: 1.00 m
```

#Örnek8:Liste

```
hizlar = [2.0, 3.5, 5.1, 6.8]

print("İlk eleman:", hizlar[0])
print("Son eleman:", hizlar[-1])
print("Dilim [1:3]:", hizlar[1:3])
print("Eleman sayısı:", len(hizlar))

hizlar[2] = 5.0
hizlar.append(8.2)
cikarilan = hizlar.pop()
```

```
print("6.8 listede var mı?:", 6.8 in hizlar)


# Aliasing (takma ad) ve bağımsız kopya
a = [1.0, 2.0]
b = a          # Referans kopyası (aynı nesne)
c = a.copy()   # Bağımsız kopya
b[0] = 99.0
print("a listesi:", a)  # [99.0, 2.0]
print("c listesi:", c)  # [1.0, 2.0]
```

ÇIKTI

```
İlk eleman: 2.0
Son eleman: 6.8
Dilim [1:3]: [3.5, 5.1]
Eleman sayısı: 4
6.8 listede var mı?: True
a listesi: [99.0, 2.0]
c listesi: [1.0, 2.0]
```

#Örnek9: Seri devrede gerilim düşümleri

```
R = [100, 220, 470, 1000]  # ohm
V = 12.0

R_seri = sum(R)
I = V / R_seri

# 1. Yol: Döngü ile
V_dusum_1 = []
for r in R:
    V_dusum_1.append(I * r)
```

```
# 2. Yol: Liste üreteci (List Comprehension)
V_dusum = [I * r for r in R]
P = [I**2 * r for r in R]
R_paralel = 1 / sum([1 / r for r in R])

print("Toplam gerilim düşümü:", round(sum(V_dusum), 2), "V")
print("En büyük direncin indeksi:", R.index(max(R)))
```

ÇIKTI

```
Toplam gerilim düşümü: 12.0 V
En büyük direncin indeksi: 3
```

#Örnek10: Konum ölçümlerinden hız ve ivme

```
serbest düşen cismin konumu (laboratuvar)
t = [0.0, 0.5, 1.0, 1.5, 2.0] # s
y = [0.0, 1.2, 4.9, 11.0, 19.6] # m
# ardışık noktalar arası ortalama hız
v = []
for i in range(1, len(t)):
    dy = y[i] - y[i-1]
    dt = t[i] - t[i-1]
    v.append(dy / dt)
    print(v) # [2.4, 7.4, 12.2, 17.2]
    ivme = (v[-1] - v[0]) / (t[-1] - t[1])
    print(round(ivme, 2)) # 9.87 ≈ g
    for ti, yi in zip(t, y):
        print(f"{ti:.1f} s → {yi:5.1f} m")
```

ÇIKTI

```
[2.4]
0.0
```

0.0 s → 0.0 m

0.5 s → 1.2 m

1.0 s → 4.9 m

1.5 s → 11.0 m

2.0 s → 19.6 m

[2.4, 7.4]

3.33

0.0 s → 0.0 m

0.5 s → 1.2 m

1.0 s → 4.9 m

1.5 s → 11.0 m

2.0 s → 19.6 m

[2.4, 7.4, 12.2]

6.53

0.0 s → 0.0 m

0.5 s → 1.2 m

1.0 s → 4.9 m

1.5 s → 11.0 m

2.0 s → 19.6 m

[2.4, 7.4, 12.2, 17.200000000000003]

9.87

0.0 s → 0.0 m

0.5 s → 1.2 m

1.0 s → 4.9 m

1.5 s → 11.0 m

2.0 s → 19.6 m

#Örnek11: Def ve Return Kullanımı

```
def kinetic_energy(m_kg, v_ms):
    if m_kg <= 0:
```

```
        raise ValueError("Kutle pozitif olmali")
    E = 0.5 * m_kg * v_ms**2
    return E
E = kinetic_energy(2.0, 3.0) # konumsal: m=2.0, v=3.0
print("E=",E) # 9.0
# isimli: sıra önemsiz
E2 = kinetic_energy(v_ms=3.0, m_kg=2.0)
for v in [1.0, 2.0, 3.0]:
    print(v, kinetic_energy(2.0, v)) # 1.0, 4.0, 9.0
```

ÇIKTI

```
E= 9.0
1.0 1.0
2.0 4.0
3.0 9.0
```

#Örnek12: Kontrolleri assert ile otomatikleştirmek

```
import math
def kinetic_energy(m_kg, v_ms):
    """Klasik kinetik enerji (J). m_kg > 0, |v_ms| << c."""
    if m_kg <= 0:
        raise ValueError("Kutle pozitif olmali")
    return 0.5 * m_kg * v_ms**2
E0 = kinetic_energy(2.0, 3.0)
assert math.isclose(E0, 9.0) # bilinen değer
assert math.isclose(kinetic_energy(2.0, 6.0), 4 * E0) # hız ölçeklemesi
assert math.isclose(kinetic_energy(4.0, 3.0), 2 * E0) # kütle ölçeklemesi
assert kinetic_energy(2.0, 0.0) == 0.0 # limit
assert kinetic_energy(2.0, -3.0) == E0 # simetri
print("Tum kontroller gecti.")
```

ÇIKTI

Tum kontroller gecti.

HAZIRLAYAN

Sümeyra Dursun

240410022